



Chapter 10:

Classes and Data Abstraction

Part II

Objectives

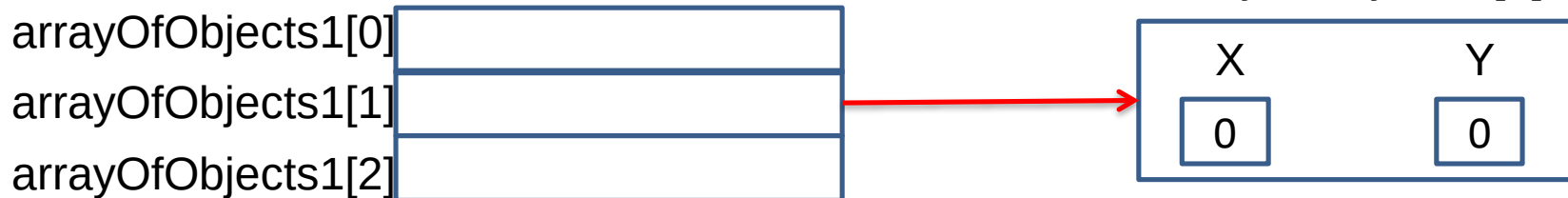
- Learn about the abstract data type (ADT)
- Explore how classes are used to implement ADTs
- Become aware of the differences between a `struct` and a `class`
- Learn about information hiding
- Explore how information hiding is implemented in C++
- Learn about the `static` members of a class

Arrays of Class Objects (Variables) and Constructors

- If you declare an array of class objects, the class should have the default constructor

```
int main()
{
    Point arrayOfObjects1[3];
    for (int i=0; i<3; i++)
        arrayOfObjects1[i].print();
}
```

(0,0)
(0,0)
(0,0)



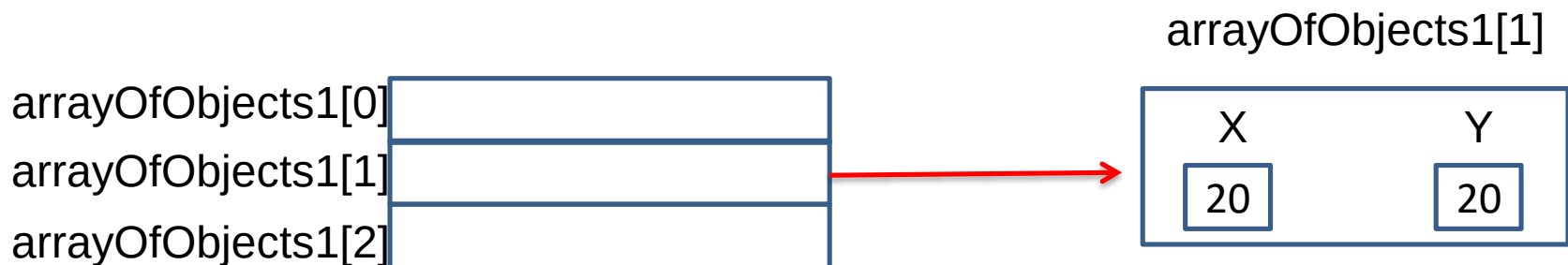
Arrays of Class Objects (Variables) and Constructors (cont'd.)

- How to initialize array of objects?

```
int main()
{
    Point arrayOfObjects2[3]={ Point(10,10), Point(20,20), Point(30,30) };
    for (int i=0; i<3; i++)
        arrayOfObjects2[i].print();
}
```

OUTPUT

```
(10,10)
(20,20)
(30,30)
```



Arrays of Class Objects (Variables) and Constructors (cont'd.)

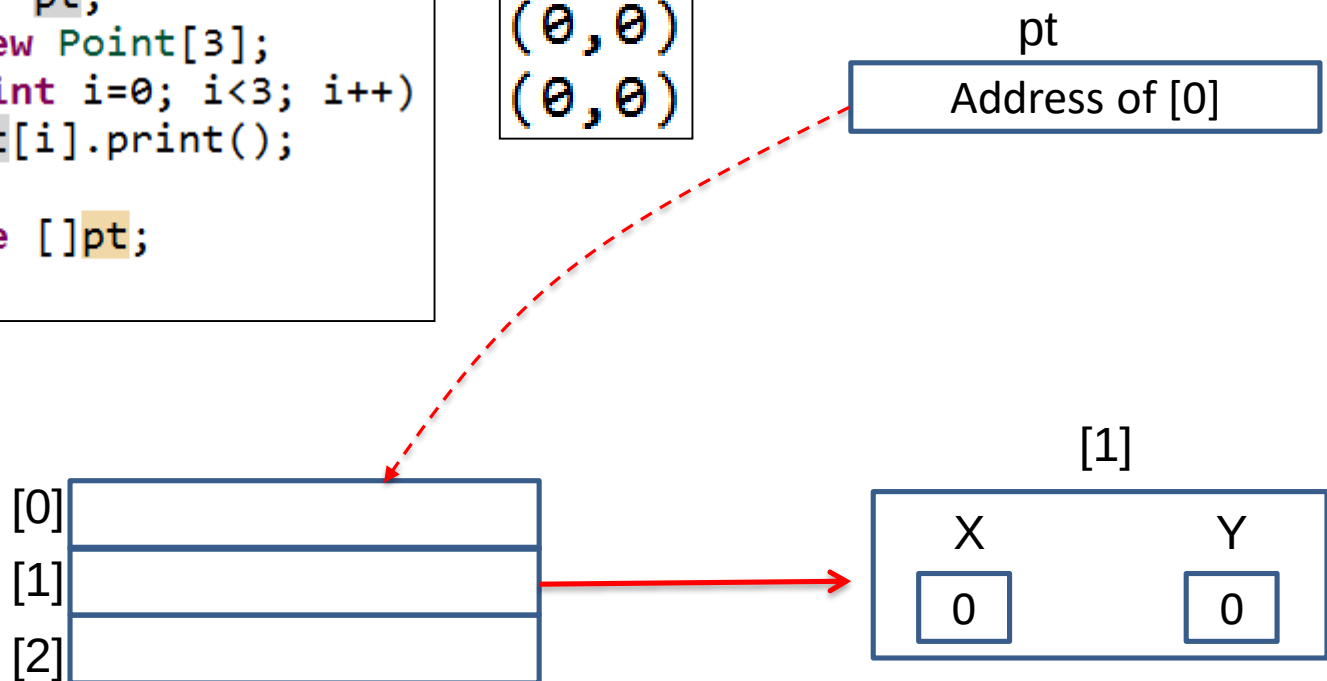
- How to create dynamic array of objects?

```
int main()
{
    Point* pt;
    pt = new Point[3];
    for (int i=0; i<3; i++)
        pt[i].print();

    delete []pt;
}
```

OUTPUT

```
(0,0)
(0,0)
(0,0)
```



The “this” Pointer

- “this” pointer is a predefined pointer that contains the address of the object referenced in the function call (current object)
- Implicitly the compiler passes to the member function a hidden pointer of that object referenced in the function call
- Implicitly the member function uses this hidden pointer to access a data member

The “this” Pointer (cont’d.)

```
int main()
{
    Point P1(10,20);
    cout<<P1.getX(); // P1.getX(&P1)
}
```

OUTPUT

10

```
int Point::getX() //accessor or getter
{
    return this->x; // equivalent to return x
    //return x;
}
```

Static Members of a Class

- There are two types of static class members
 - **Static data members**
 - **Static member functions**

```
class Test
{
public:
    static int count; //Definition of static a data member
    static int getCount(){return count;}
};
```


Static Members of a Class (cont'd.)

Static data members

- Static data members must be declared outside the class as global data item.
 - You may initialize it , otherwise it would be initialized to **0**

```
class Test
{
public:
    static int count; //Definition of static a data member
    static int getCount(){return count;}
};

int Test::count=0;
```

- **private**, **public** and **protected** static data members must be declared outside the class

Static Members of a Class (cont'd.)

Static data members

- Static data members exist independently of any class object
 - You can access it using the class name and binary resolution operator (::) without object name

```
int main()
{
    Test::count=1;
    cout<<Test::count<<endl;
    Test::count++;
}
```

OUTPUT

1

- A static data member is a **hybrid** between a **global variable** and a **data member** belonging to a class

Static Members of a Class (cont'd.)

Static data members

- Multiple objects of a class each have their own copy of non-static member variables
- All objects of a class share any static member of the class

Static Members of a Class (cont'd.)

Static member functions

- Static member functions are functions associated with a class not an object, and have the following characteristics:

1. Client code can call these function using class name and (::)

```
int main()
{
    int c= Test::getCount();
}
```

2. These function can directly access only static data members and static member functions belonging to that class
3. These functions do not have “this” pointer (why?)

- Note: you can call a static member function using object name.

```
int main()
{
    int c,b;
    Test t;
    b= t.getCount();
}
```

Copy Constructor

- A copy constructor is used to initialize a class object by using the values of an existing object of the same type (member-to-member copy)
- Copy constructor is provided by the compiler
- For classes with pointer member variables, the user must write the copy constructor:

Copy Constructor (cont'd.)

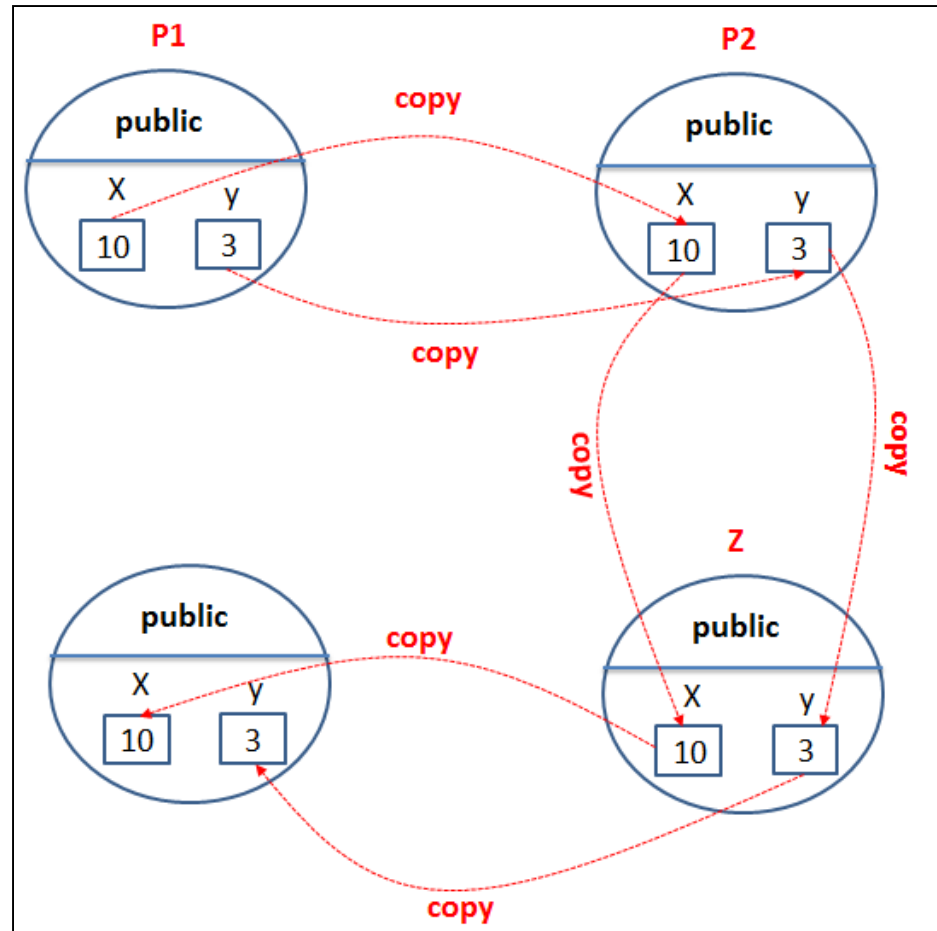
- A copy constructor automatically executes in three situations:
 - When an object is declared and initialized by using the value of another object
 - When an object is passed by value as a parameter
 - When the return value of a function is an object

Copy Constructor (cont'd.)

```
.....  
Point Test(Point Z)  
{  
    return Z;  
}  
  
int main()  
{  
    Point P1(10,3);  
    Point P2=P1;  
    P1=Test(P2);  
  
    P1.print();  
    P2.print();  
}
```

OUTPUT

(10,3)
(10,3)



Copy Constructor (cont'd.)

```
.....
Point::Point(Point& in)
{
    x=in.x;
    y=in.y;
    cout<<"Copy Constructor"<<endl;
}

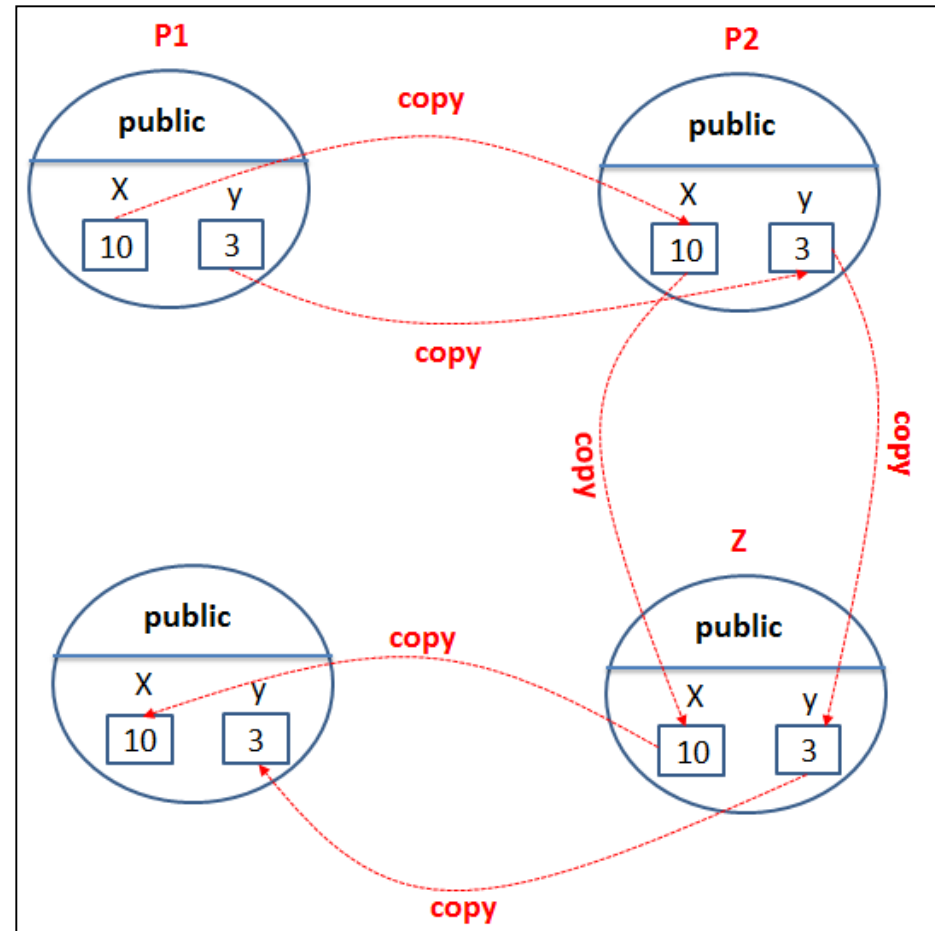
Point Test(Point Z)
{
    return Z;
}

int main()
{
    Point P1(10,3);
    Point P2=P1;
    P1=Test(P2);

    P1.print();
    P2.print();
}
```

OUTPUT

```
(10,3)
(10,3)
```



Copy Constructor (cont'd.)

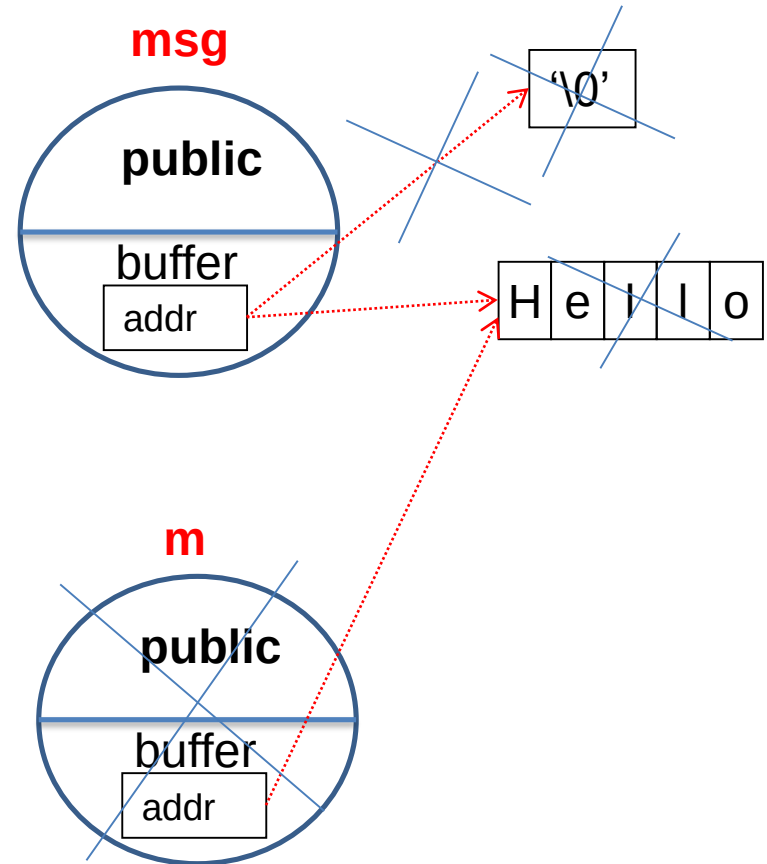
```
class Message
{
public:
    Message() { buffer=new char('\0'); }
    void dispaly () { cout<<buffer<<endl; }
    void Set(char* str)
    {
        delete []buffer;
        buffer = new char[strlen(str)+1];
        strcpy(buffer, str);
    }
    ~Message(){ delete []buffer; }
private:
    char* buffer; // to store message string
};

void Show(Message m)
{ m.dispaly(); }

int main()
{
    Message msg;
    msg.Set("Hello");
    Show(msg);
    msg.dispaly();
}
```

OUTPUT

Hello
Runtime
Error



Copy Constructor (cont'd.)

```
class Message
{
public:
    Message() { buffer=new char('\0'); }
    Message (Message& in)
    {   buffer =new char[strlen(in.buffer)+1];
        strcpy(buffer,in.buffer);
    }
    void dispaly () { cout<<buffer<<endl; }

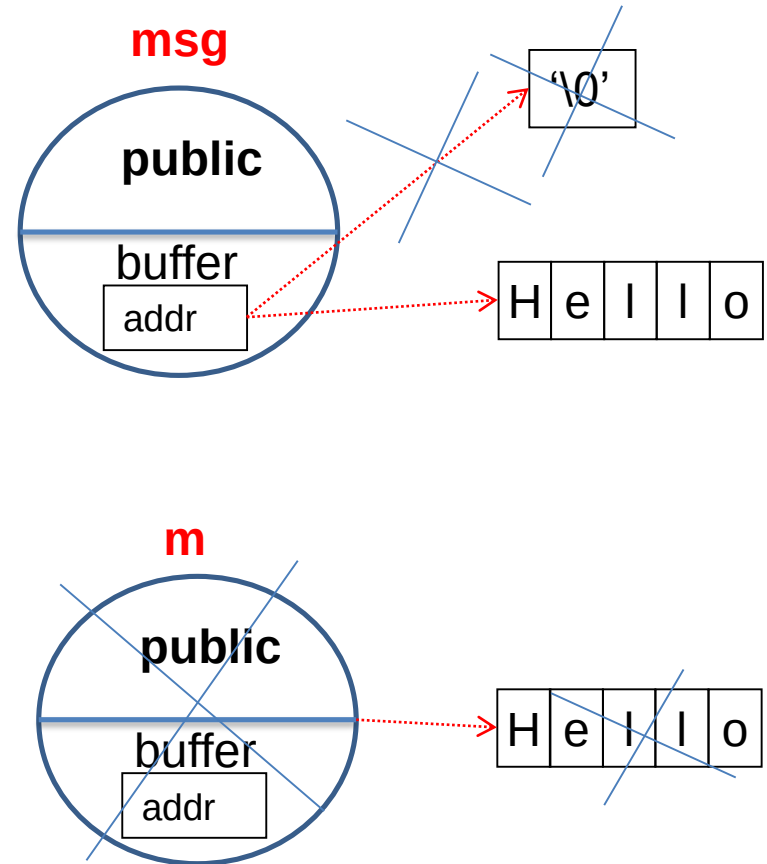
    void Set(char* str)
    {   delete []buffer;
        buffer = new char[strlen(str)+1];
        strcpy(buffer, str);
    }
    ~Message(){ delete []buffer; }
private:
    char* buffer; // to store message string
};

void Show(Message m)
{   m.dispaly(); }

int main()
{   Message msg;
    msg.Set("Hello");
    Show(msg);
    msg.dispaly();}
```

OUTPUT

Hello
Hello



Data Abstract, Classes, and Abstract Data Types

- Abstraction
 - Separating design details from usage
 - Separating the logical properties from the implementation details
- Abstraction can also be applied to data
- Abstract data type (ADT): data type that separates the logical properties from the implementation details

A struct Versus a class

- By default, members of a `struct` are `public`
 - `private` specifier can be used in a `struct` to make a member private
- By default, the members of a `class` are `private`
- `classes` and `structs` have the same capabilities

A struct Versus a class (cont'd.)

- In C++, the definition of a `struct` was expanded to include member functions, constructors, and destructors
- If all member variables of a `class` are `public` and there are no member functions
 - Use a `struct`

The End